

UNIVERSITY OF LONDON

BSc Computer Science

CM2035 Algorithms and Data Structures 2

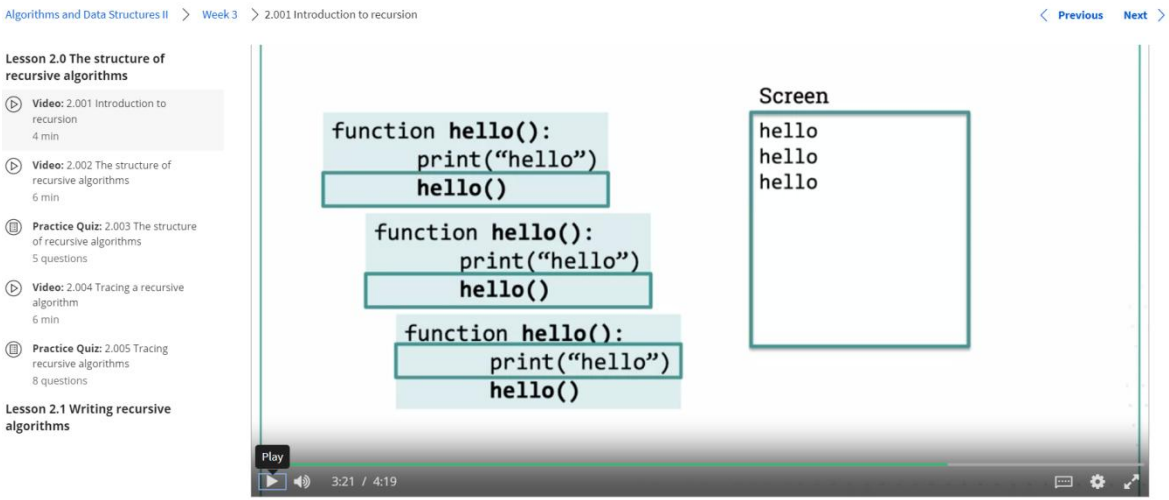
Coursera Wk 3/4 (180min)

Objectives:

Trace and write recursive algorithms

Write the recursive version of an iterative algorithm using pseudocode

Calculate the time complexity of recursive algorithms.

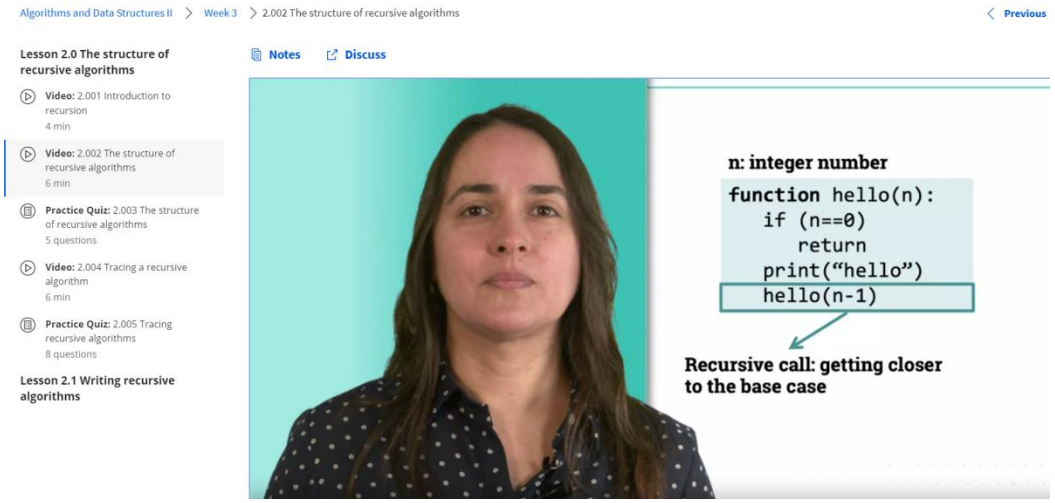



```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
using namespace std;

int   sum   (int k)
{
  if (k > 0) // stopping case = base case (infinite recursion)
  {
    return k + sum(k - 1);
  } else
    {
       return 0;
    }
}

int main() {
  int result = sum(4);
  cout << result;
  return 0;
}
// Convert the above program to recur 3 Hellos
```

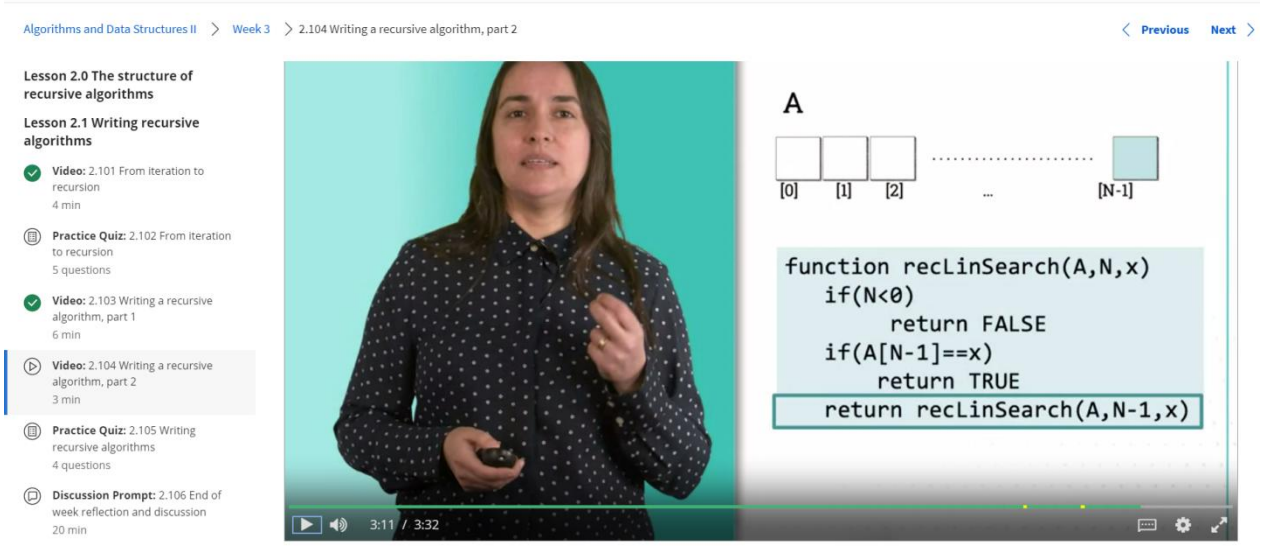





```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
using namespace std;

int sum(int k)
{
  if (k > 0)
 {
    return k + sum(k - 1);
  } else
   {
    return 0;
   }
}

int main() {
  int result = sum(4);
  cout << result;
  return 0;
}
// WHERE IS THE BASE CASE?
// WHERE IS THE RECURSIVE CALL?
```



	[0]	[1]	[2]	[3]	[4]	[5]
A	10	20	30	40	50	60

N	6
X	30

```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>

using namespace std;

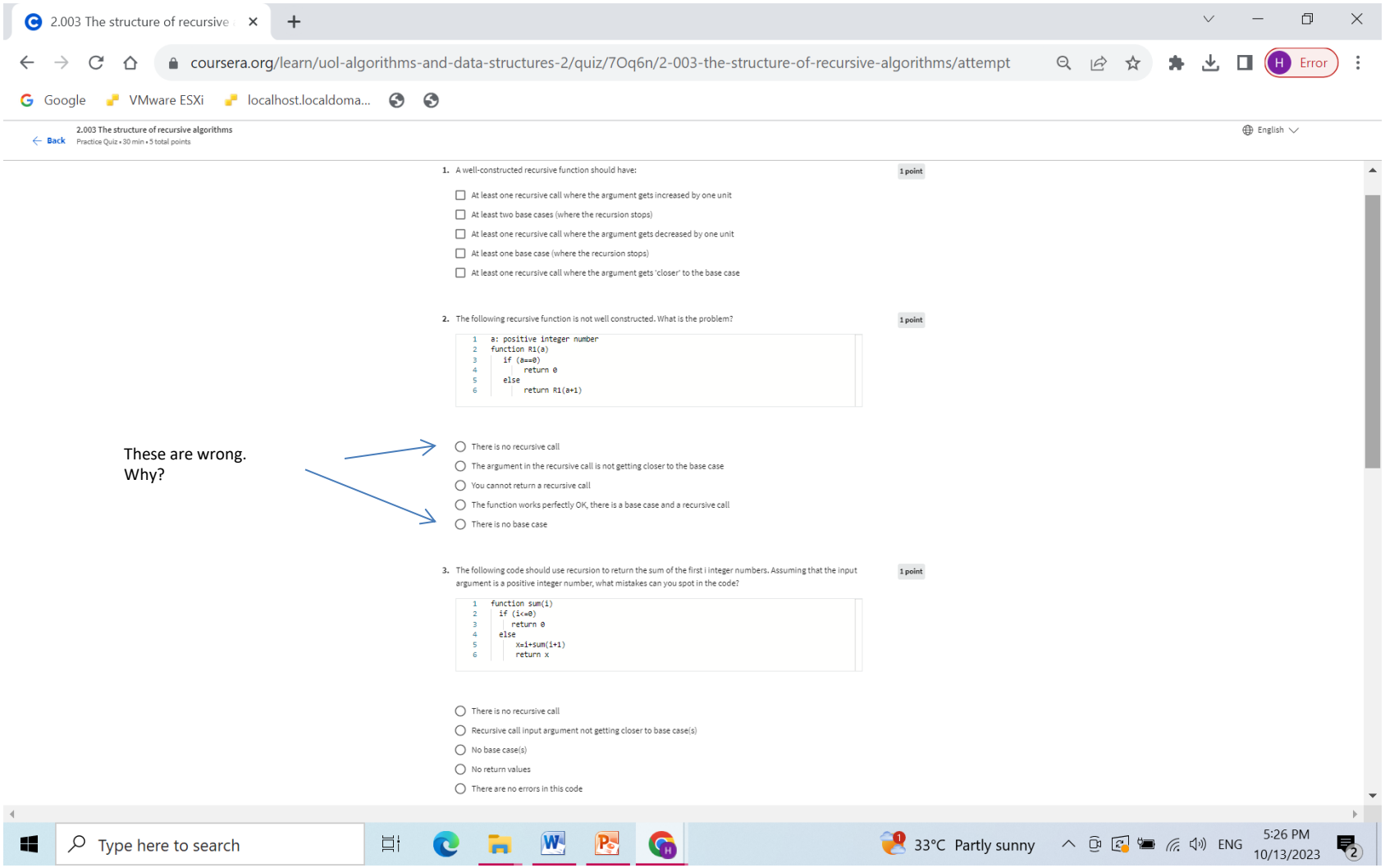
// Recursive function to search for x in arr[]
int searchElement(int arr[], int size, int x) {

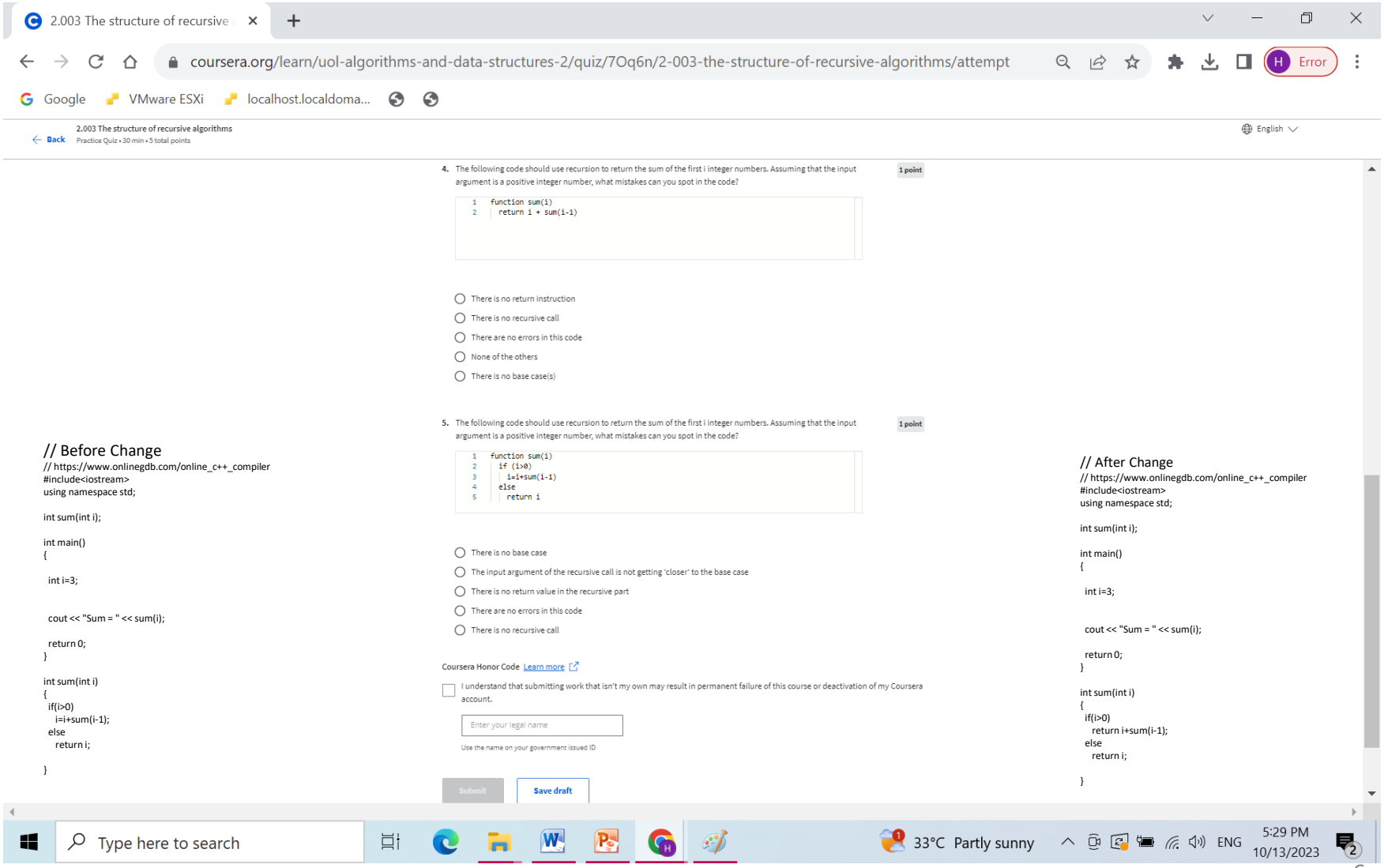
   size--;
   cout<<" " << arr[size];
   // Base case (Element not present in the array)
   if (size < 0) { // stop
      return -1;
   }
   // Base case (Element found, return its position)
   if (arr[size] == x) { // stop
      return size;
   }

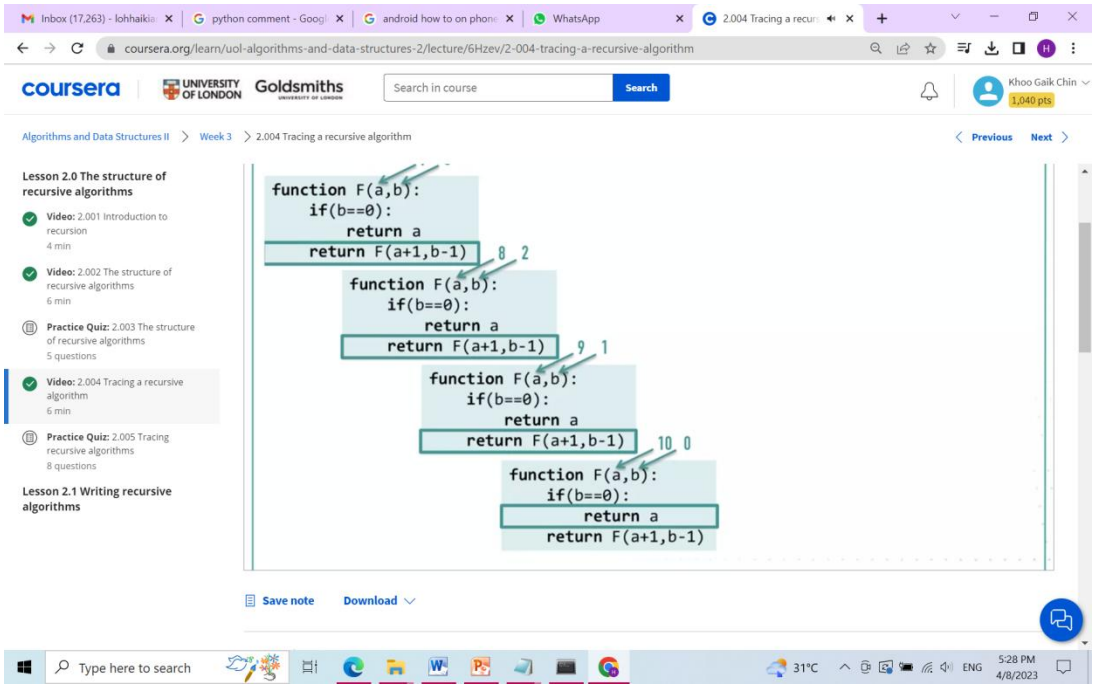
   // Recursive case
   return searchElement(arr, size, x);

}


// Driver code
int main() {
   int arr[] = {17, 15, 11, 8, 13, 19};
   int size = sizeof(arr) / sizeof(arr[0]);
   int x = 11;
   int idx = searchElement(arr, size, x);
   if (idx != -1)
      cout << "Element " << x << " is present at index " <<idx;
   else
      cout << "Element " << x << " is not present in the array";
   return 0;
}
```


2.003 The structure of recursive
coursera.org/learn/uol-algorithms-and-data-structures-2/quiz/7Oq6n/2-003-the-structure-of-recursive-algorithms/attempt
Error
Google
VMware ESXi
localhost.localdoma...
Back
2.003 The structure of recursive algorithms
Practice Quiz • 30 min • 5 total points
English
1. A well-constructed recursive function should have:
1 point
At least one recursive call where the argument gets increased by one unit
At least two base cases (where the recursion stops)
At least one recursive call where the argument gets decreased by one unit
At least one base case (where the recursion stops)
At least one recursive call where the argument gets 'closer' to the base case
2. The following recursive function is not well constructed. What is the problem?
1 point
a: positive integer number
function R1(a)
if (a==0)
return 0
else
return R1(a+1)
These are wrong.
Why?
There is no recursive call
The argument in the recursive call is not getting closer to the base case
You cannot return a recursive call
The function works perfectly OK, there is a base case and a recursive call
There is no base case
3. The following code should use recursion to return the sum of the first i integer numbers. Assuming that the input argument is a positive integer number, what mistakes can you spot in the code?
1 point
function sum(i)
if (i<=0)
return 0
else
x=i+sum(i+1)
return x
There is no recursive call
Recursive call input argument not getting closer to base case(s)
No base case(s)
No return values
There are no errors in this code
Type here to search
33°C Partly sunny
ENG
5:26 PM
10/13/2023

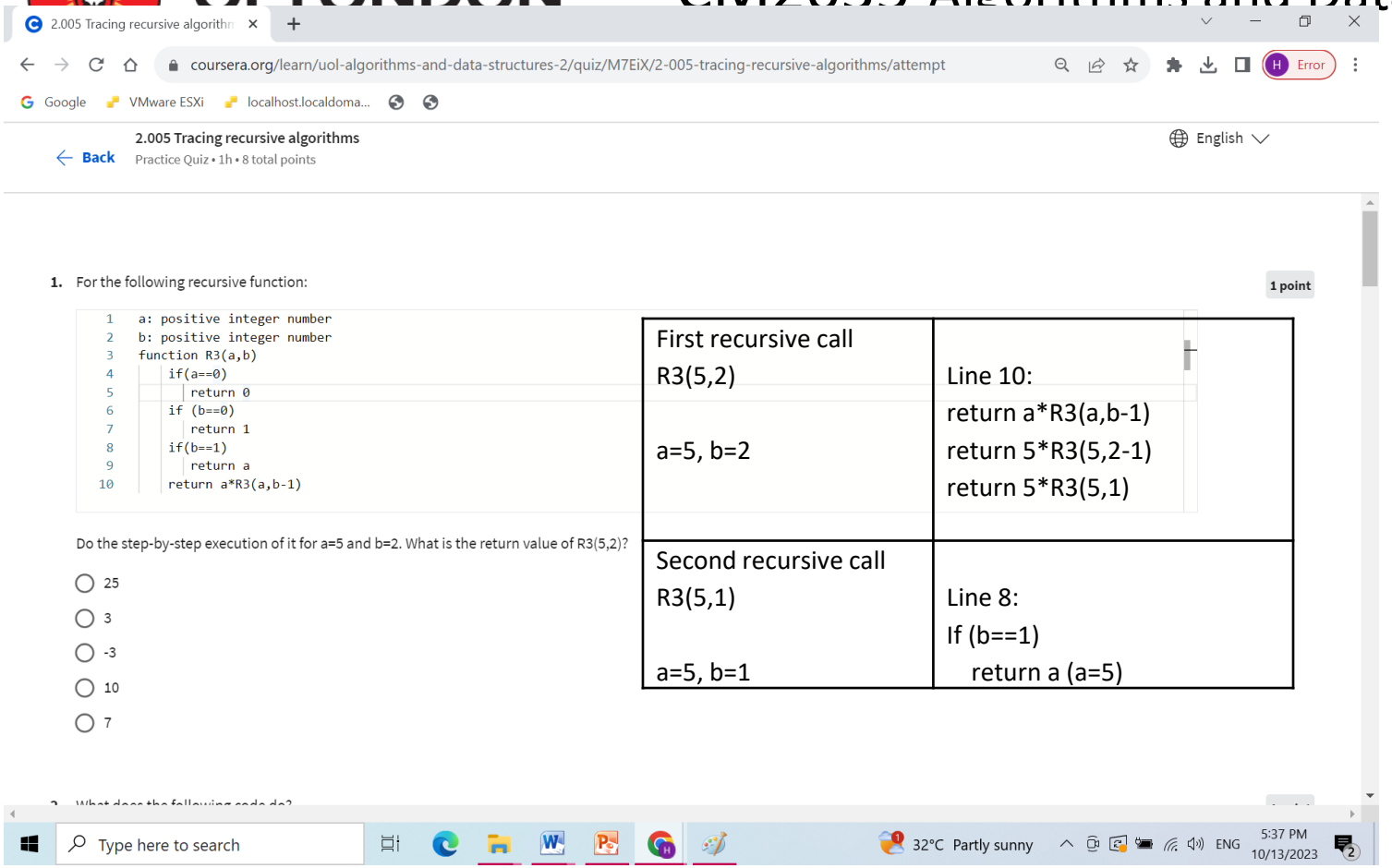

2.003 The structure of recursive
coursera.org/learn/uol-algorithms-and-data-structures-2/quiz/7Oq6n/2-003-the-structure-of-recursive-algorithms/attempt
Google
VMware ESXi
localhost.localdoma...
Back
2.003 The structure of recursive algorithms
Practice Quiz • 30 min • 5 total points
English
4. The following code should use recursion to return the sum of the first i integer numbers. Assuming that the input argument is a positive integer number, what mistakes can you spot in the code?
1 point
1 function sum(i)
2 return i + sum(i-1)
There is no return instruction
There is no recursive call
There are no errors in this code
None of the others
There is no base case(s)
5. The following code should use recursion to return the sum of the first i integer numbers. Assuming that the input argument is a positive integer number, what mistakes can you spot in the code?
1 point
1 function sum(i)
2 if (i>0)
3 i=i+sum(i-1)
4 else
5 return i
There is no base case
The input argument of the recursive call is not getting 'closer' to the base case
There is no return value in the recursive part
There are no errors in this code
There is no recursive call
Coursera Honor Code Learn more
I understand that submitting work that isn't my own may result in permanent failure of this course or deactivation of my Coursera account.
Enter your legal name
Use the name on your government issued ID
Submit
Save draft
// Before Change
// https://www.onlinegdb.com/online_c++_compiler
#include<iostream>
using namespace std;
int sum(int i);
int main()
{
int i=3;
cout << "Sum = " << sum(i);
return 0;
}
int sum(int i)
{
if(i>0)
i=i+sum(i-1);
else
return i;
}
// After Change
// https://www.onlinegdb.com/online_c++_compiler
#include<iostream>
using namespace std;
int sum(int i);
int main()
{
int i=3;
cout << "Sum = " << sum(i);
return 0;
}
int sum(int i)
{
if(i>0)
return i+sum(i-1);
else
return i;
}
Type here to search
33°C Partly sunny
ENG
5:29 PM
10/13/2023



Question:
If we call the function with:

F (7,3)

How many times the function F recur?




```
// https://www.onlinegdb.com/online_c++_com
#include<iostream>
using namespace std;

int R3(int a, int b);

int main() {

 int a=5, b=2;

 cout << R3(a,b);

 return 0;
}

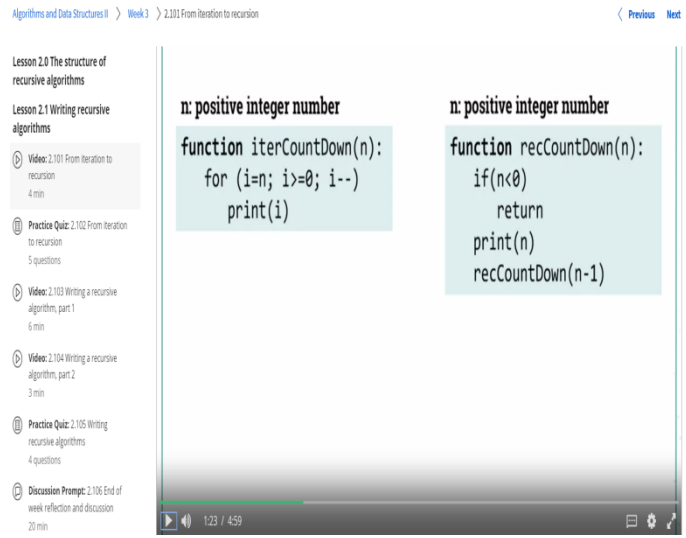
int R3(int a, int b)
{
 if(a==0)
  return 0;
 if(b==0)
  return 1;
 if(b==1)
  return a;
 return a*R3(a,b-1);
}
```

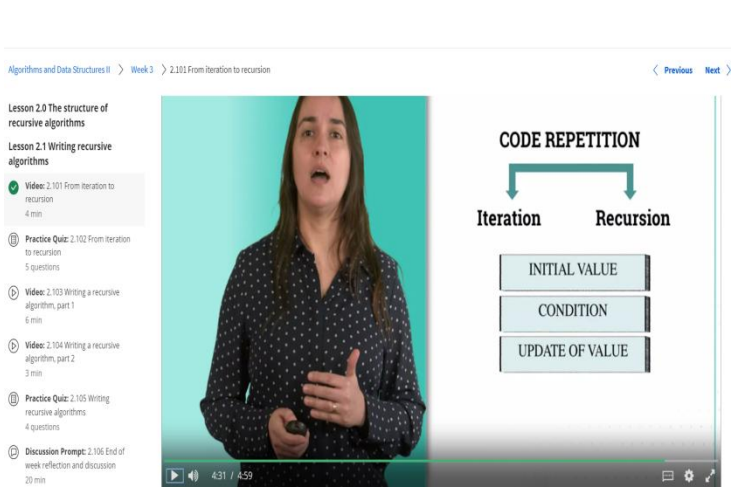
UNIVERSITY OF LONDON

BSc Computer Science

CM2035 Algorithms and Data Structures 2

ITERATION VS RECURSION






```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
using namespace std;

int main() {
   int n;
   long factorial = 1.0;

   cout << "Enter a positive integer: ";
   cin >> n;

   if (n < 0)
      cout << "Error! Factorial of a negative number doesn't exist.";
   else {
      for(int i = 1; i <= n; ++i) {
         factorial *= i;
      }
      cout << "Factorial of " << n << " = " << factorial;
   }

   return 0;
}
```

```
// https://www.onlinegdb.com/online_c++_compiler
#include<iostream>
using namespace std;

int factorial(int n);

int main() {

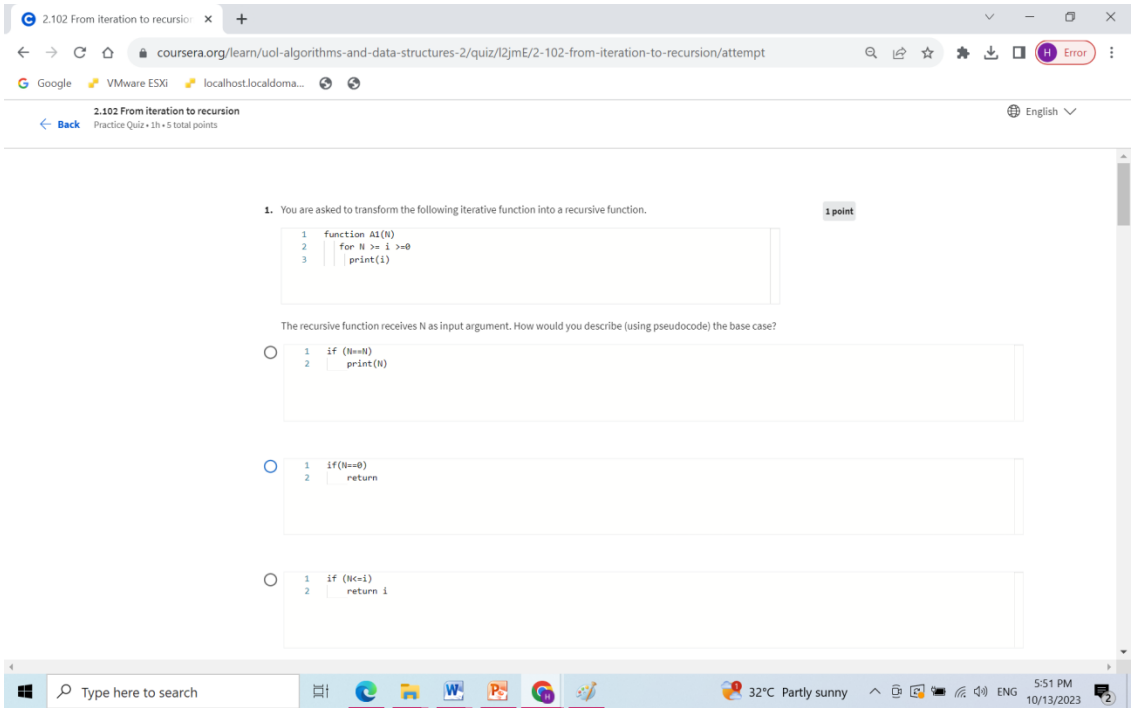
 int n;

 cout << "Enter a positive integer: ";
 cin >> n;

 cout << "Factorial of " << n << " = " << factorial(n);

 return 0;
}

int factorial(int n) {
 if(n > 1)
  return n * factorial(n - 1);
 else
  return 1;
}
```




```
//
https://www.onlinegdb.com/online_c++_compiler
#include<iostream>
using namespace std;

void A1(int N);

int main()
{

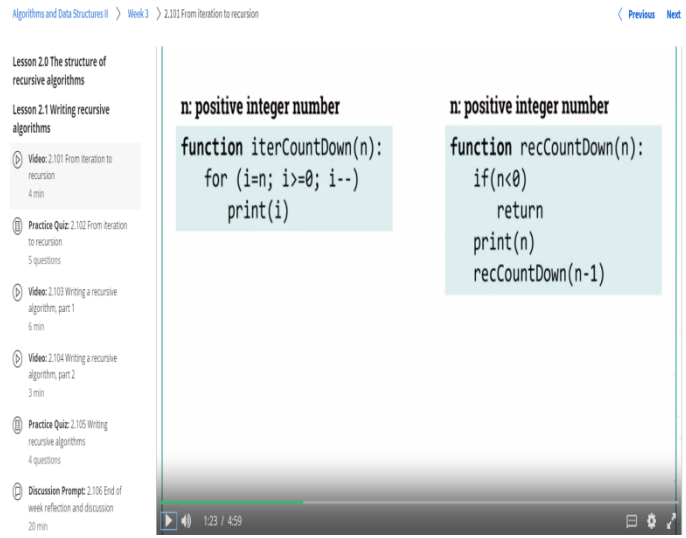
  int N=3;

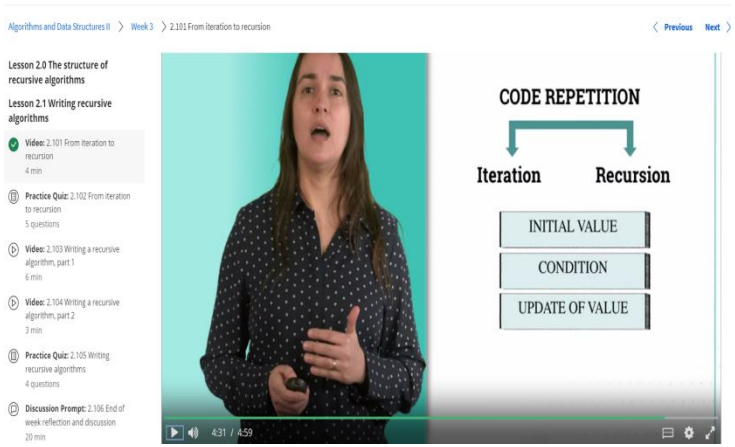

  A1(N);

  return 0;
}

void A1(int N)
{
  if(N==0)
    cout << N;
  else
    { cout << N;
      A1(N-1);
    }
}
```

ITERATION VS RECURSION






```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
using namespace std;

int main() {
   int n;
   long factorial = 1.0;

   cout << "Enter a positive integer: ";
   cin >> n;

   if (n < 0)
      cout << "Error! Factorial of a negative number doesn't exist.";
   else {
      for(int i = 1; i <= n; ++i) {
         factorial *= i;
      }
      cout << "Factorial of " << n << " = " << factorial;
   }

   return 0;
}
```

```
// https://www.onlinegdb.com/online_c++_compiler
#include<iostream>
using namespace std;

int factorial(int n);

int main() {

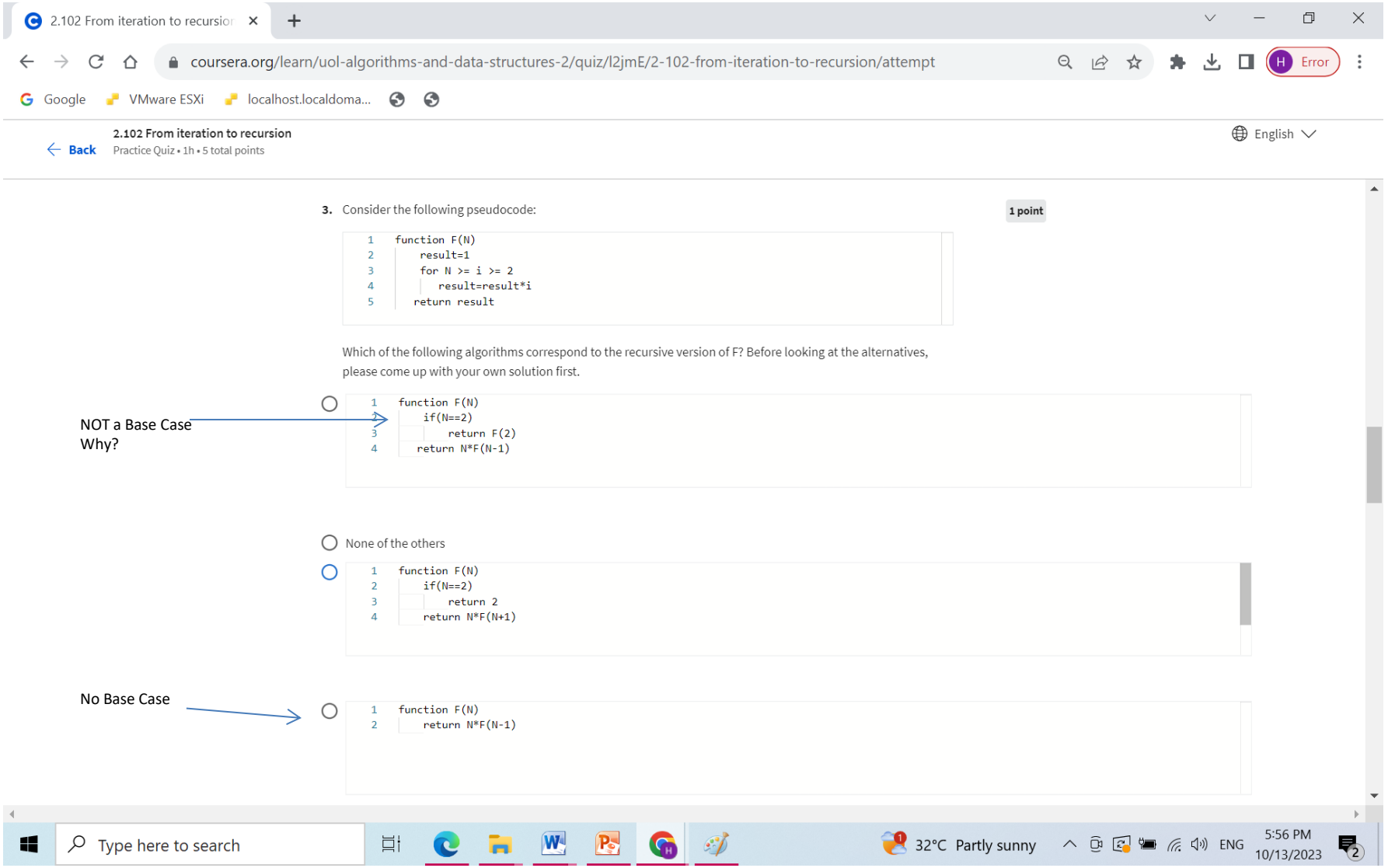
 int n;

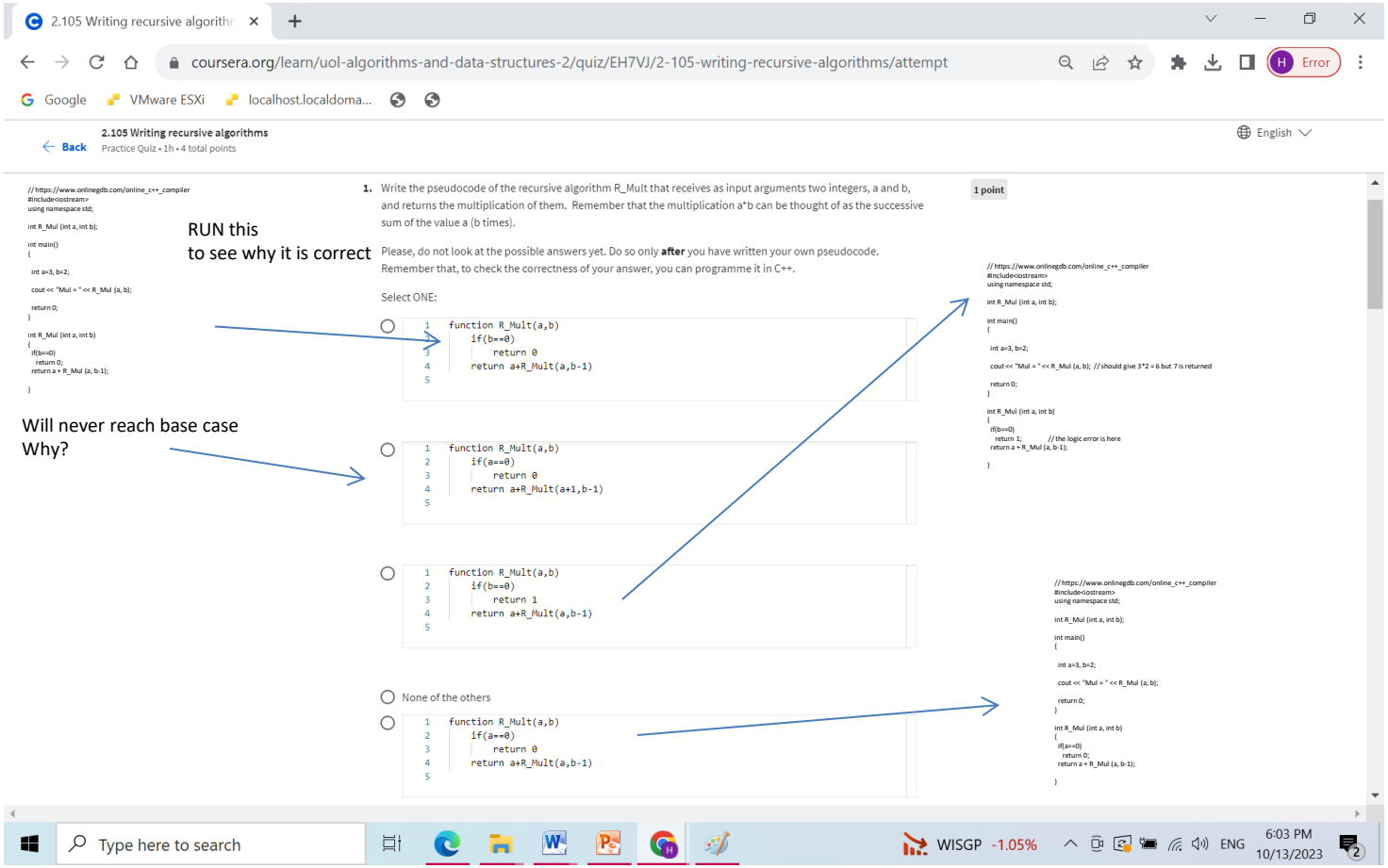
 cout << "Enter a positive integer: ";
 cin >> n;

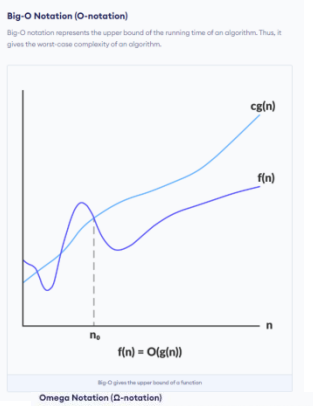
 cout << "Factorial of " << n << " = " << factorial(n);

 return 0;
}

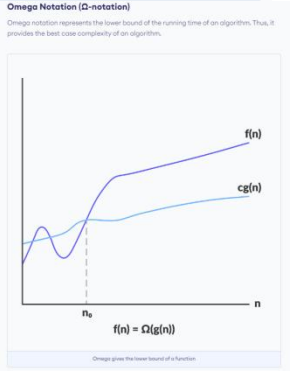
int factorial(int n) {
 if(n > 1)
  return n * factorial(n - 1);
 else
  return 1;
}
```


2.102 From iteration to recursion
coursera.org/learn/uol-algorithms-and-data-structures-2/quiz/l2jmE/2-102-from-iteration-to-recursion/attempt
Google
VMware ESXi
localhost.localdoma...
Back
2.102 From iteration to recursion
Practice Quiz • 1h • 5 total points
English
3. Consider the following pseudocode:
1 point
function F(N)
result=1
for N >= i >= 2
result=result*i
return result
Which of the following algorithms correspond to the recursive version of F? Before looking at the alternatives, please come up with your own solution first.
NOT a Base Case
Why?
function F(N)
if(N==2)
return F(2)
return N*F(N-1)
None of the others
function F(N)
if(N==2)
return 2
return N*F(N+1)
No Base Case
function F(N)
return N*F(N-1)
Type here to search
32°C Partly sunny
ENG
5:56 PM
10/13/2023

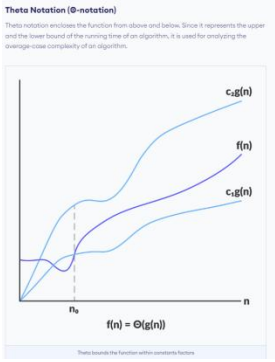

2.105 Writing recursive algorithm
coursera.org/learn/uol-algorithms-and-data-structures-2/quiz/EH7VJ/2-105-writing-recursive-algorithms/attempt
Back
2.105 Writing recursive algorithms
Practice Quiz • 1h • 4 total points
English
1 point
1. Write the pseudocode of the recursive algorithm R_Mult that receives as input arguments two integers, a and b, and returns the multiplication of them. Remember that the multiplication a*b can be thought of as the successive sum of the value a (b times).
Please, do not look at the possible answers yet. Do so only after you have written your own pseudocode. Remember that, to check the correctness of your answer, you can programme it in C++.
Select ONE:
RUN this
to see why it is correct
Will never reach base case
Why?
function R_Mult(a,b)
if(b==0)
return 0
return a+R_Mult(a,b-1)
function R_Mult(a,b)
if(a==0)
return 0
return a+R_Mult(a+1,b-1)
function R_Mult(a,b)
if(b==0)
return 1
return a+R_Mult(a,b-1)
None of the others
function R_Mult(a,b)
if(a==0)
return 0
return a+R_Mult(a,b-1)
Type here to search
WISGP -1.05%
ENG
6:03 PM
10/13/2023



What is Big O O(n^2) **? e.g. {3,2,1} n^2 = 3^2 = 9 iterations**
Big O describes the upper bound of an algorithm.
Using our Insertion Sort example, the rate of growth of our algorithm is *at most* quadratic, or O(n^2). **Worst-Case scenario**



What is Big Omega *Ω(n)* **? e.g. {1,2,3} n = 3 iterations**
Big Omega describes the lower bound of an algorithm.
Using our Insertion Sort example, if *the input is already sorted*, then the rate of growth of our algorithm is *at least* linear, or Ω(n).
We can also think of this as our **best-case scenario.**



What is Big Theta *Θ(n) or ϑ(n)* **? n^2 = 3^2 = 9 iterations**
Using our Insertion Sort example, we know that the rate of growth is *at most* O(n^2) and *at least* Ω(n).
Big Theta is often used to describe the average, or expected, case for an algorithm. So for Insertion Sort, the **average case** for Big Theta is *n^2*.

https://www.programiz.com/dsa/asymptotic-notations

https://jarednielsen.com/big-o-omega-theta/

	[0]	[1]	[2]	[3]	[4]
A	10	20	30	40	50

N = 5

x = 10

(a) Consider the following algorithm:

```
A: array of positive integer numbers
N: number of elements in A

function F1(A,N)
          if(N==0):
               return -1
          x=A[0]
          for (0 < i < N)
               if(A[i]>x):
                    x=A[i]
          return x
end function
```

Assume that the array A is equal to [3,2,1,4]. What value is returned by F1(A,4)? [4]

Algorithm	Best Time Complexity	Worst Time Complexity
Linear Search	O(1)	O(n)
Binary Search	O(1)	O(log n)
Bubble Sort	O(n)	O(n^2)
Selection Sort	O(n^2)	O(n^2)
Insertion Sort	O(n)	O(n^2)
Merge Sort	O(nlogn)	O(nlogn)
Quick Sort	O(nlogn)	O(n^2)
Heap Sort	O(nlogn)	O(nlogn)
Bucket Sort	O(n+k)	O(n^2)
Radix Sort	O(nk)	O(nk)
Tim Sort	O(n)	O(nlogn)
Shell Sort	O(n)	O((nlog(n))^2)

(c) Consider the following algorithm:

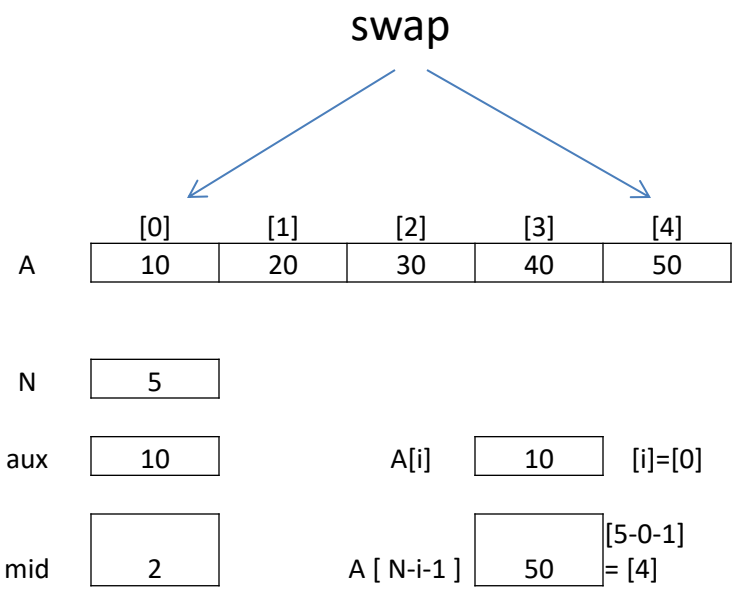
```
A: array of integer numbers
N: number of elements in A

function F2(A,N)
    if(N==0):
          return A
    mid=floor(N/2)
    for 0 <= i < mid
              aux=A[i]
              A[i]=A[N-i-1]
              A[N-i-1]=aux
    return A
end function
```

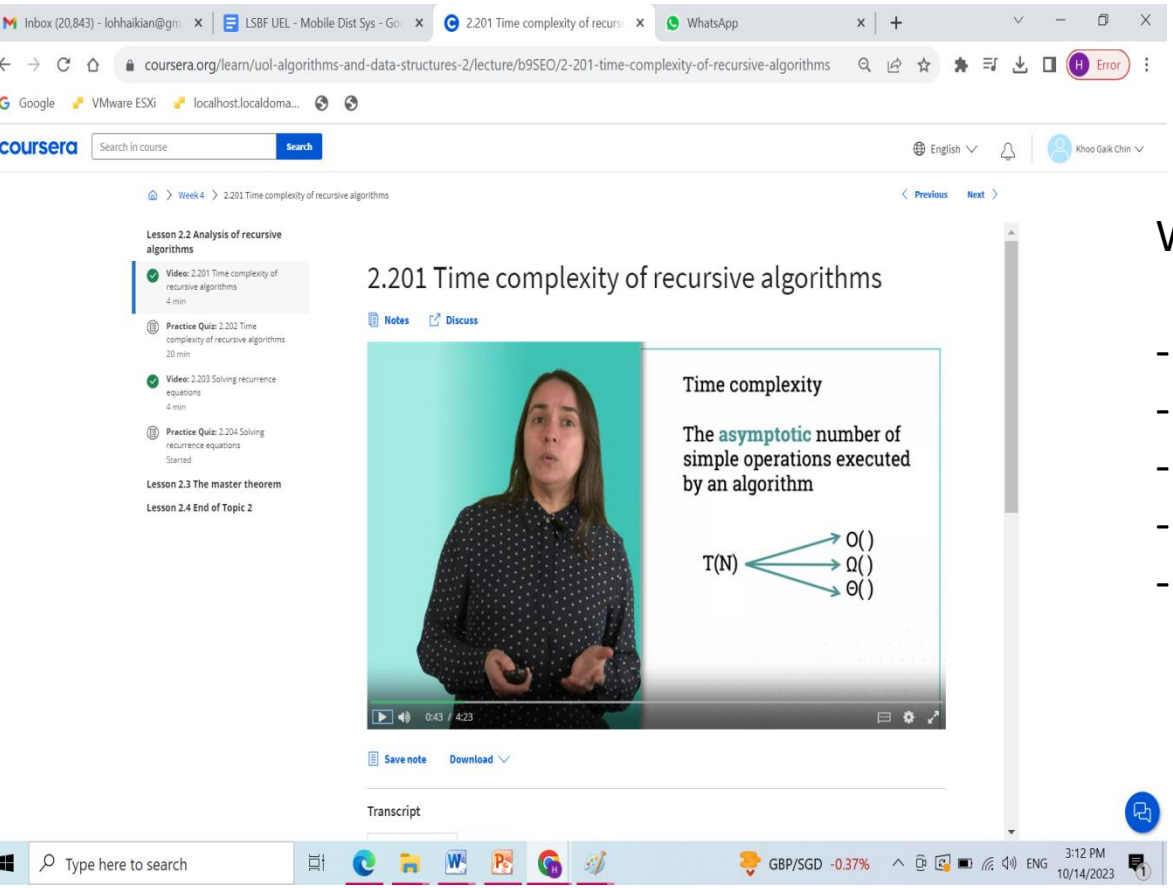
The operation floor(x) returns the largest integer smaller than or equal to x.

Select ALL statements that apply. [4]

i. The worst-case time complexity of F2 is Θ(N)
ii. The worst and best case time complexity of F2 are the same
iii. The worst-case time complexity of F2 is Θ(log N)
iv. The best-case time complexity of F2 is Θ(1)

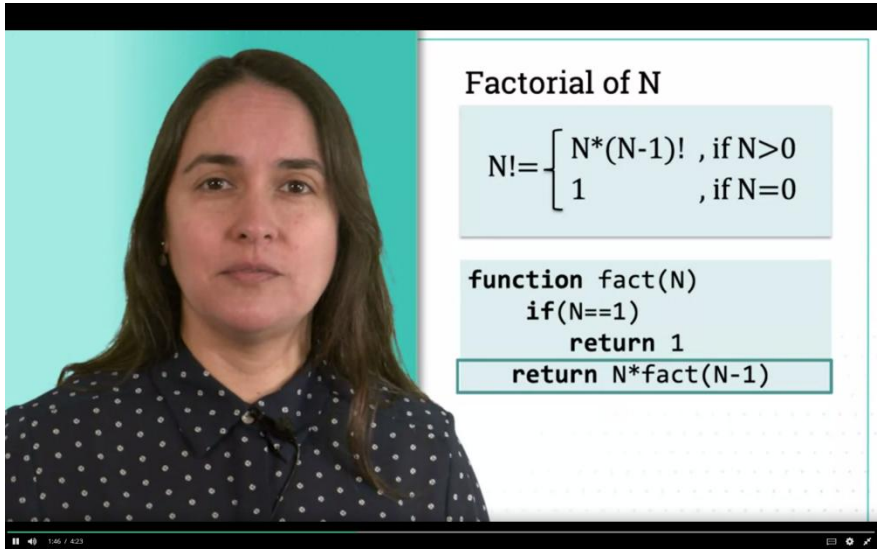


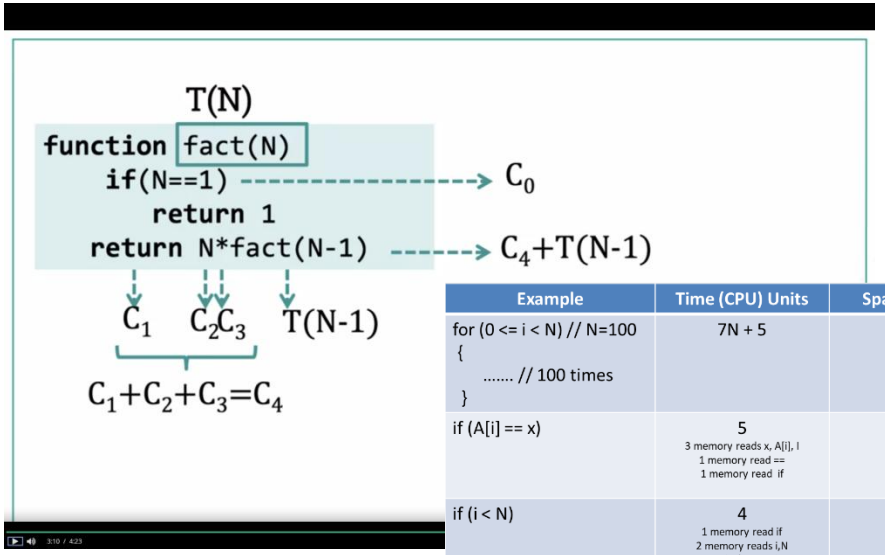
Note :
If O(N/2), we say it is O(N)
as N grows very big



Why are we studying this?

- Measure the efficiency of the algorit
- In terms of Time (CPU)
- = O(N) = iterations / data / arrayleng
- = Linear Search
- = iterations/recursion

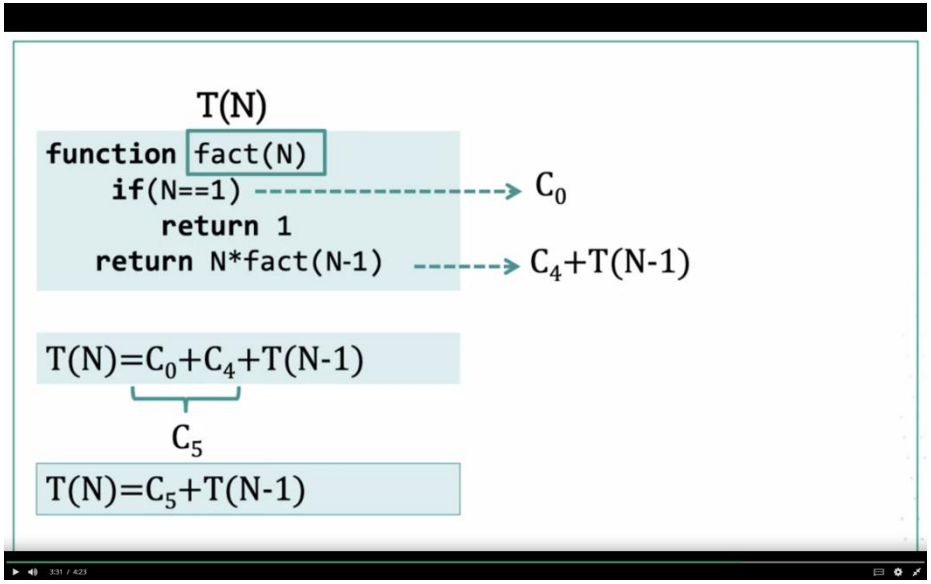




Example	Time (CPU) Units	Space (memory)

for (0 <= i < N) // N=100
{
..... // 100 times
}
	7N + 5	

if (A[i] == x)	5 3 memory reads x, A[i], I 1 memory read == 1 memory read if	
if (i < N)	4 1 memory read if 2 memory reads i,N 1 numerical operation <	
i = i + 1	3 1 memory read i 1 numerical operation +1 1 memory write i	
return i	1	
max = a	2 1 memory read a 1 memory write max	1



$$T(N)=C_5+T(N-1)$$

Recurrence equation

Inbox (20,843) - lohhaikian@gm | LSBF UEL - Mobile Dist Sys - Go | 2.202 Time complexity of recursi | WhatsApp

coursera.org/learn/uol-algorithms-and-data-structures-2/quiz/EVebz/2-202-time-complexity-of-recursive-algorithms/attempt

Google | VMware ESXi | localhost.localdoma...

Back

2.202 Time complexity of recursive algorithms

Practice Quiz • 20 min • 3 total points

English

1. What is the recurrence equation that describes the running time of the algorithm below?

```
1  A: 1D array of N integer numbers
2  B: 1D array of N integer numbers
3  N: number of elements of arrays A and B
4  function ALG1(A,B,N):
5      if(N==0):
6          return 0
7      return A[N-1]*B[N-1]+ALG1(A,B,N-1)
```

Recurring

Not Recurring

- ○ T(N)=T(N+1)+C
- ○ None of the others
- ○ T(N)=2*T(N-1)+C
- ○ T(N)=T(N/2)+C
- ○ T(N)=T(N-1)+C

Example	Time (CPU) Units	Space
for (0 <= i < N) // N=100 { ..... // 100 times }	7N + 5	
if (A[i] == x)	5 (3 memory reads x, A[i], i; 1 memory read ==; 1 memory read if)	
if (i < N)	4 (1 memory read if; 2 memory reads i,N; 1 numerical operation <)	
i = i + 1	3 (1 memory read i; 1 numerical operation +1; 1 memory write i)	
return i	1	
max = a	2 (1 memory read a; 1 memory write max)	

T(N)

function fact(N)

if(N==1) ----------> C_0

return 1

return N*fact(N-1) ----------> $C_4+T(N-1)$

$T(N)=C_0+C_4+T(N-1)$

C_5

$T(N)=C_5+T(N-1)$

$T(N)=C_5+T(N-1)$

Recurrence equation

3:31 / 4:23

Type here to search

GBP/SGD -0.37% ENG 3:31 PM 10/14/2023

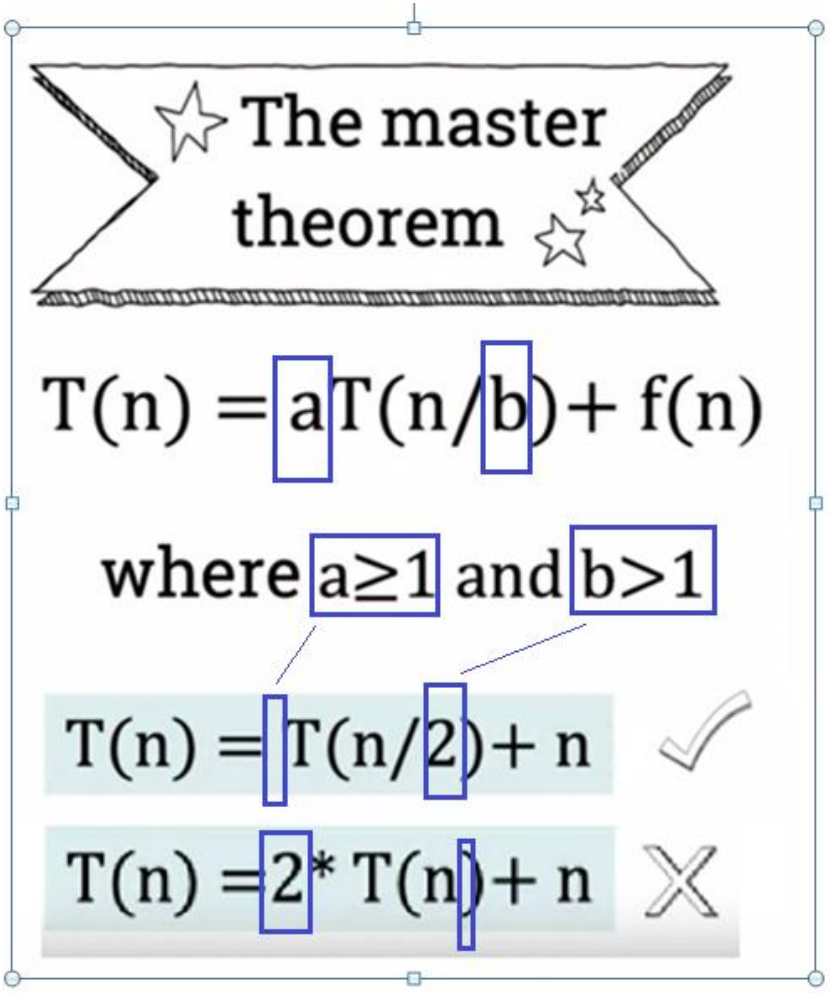
2.202 Time complexity of recursive algorithms

Back

Practice Quiz • 20 min • 3 total points

```
1   A: 1D array
2   low: lowest index
3   high: highest index
4
5   function ALG2(A,low,high):
6       if(high-low==1):
7           if(A[low]<A[high])
8               return high
9           else
10              return low
11      if (low==high)
12          return high
13      mid=low+floor((high-low)/2)
14      a=ALG2(A,low,mid)
15      b=ALG2(A,mid+1,high)
16      if(A[a]>A[b])
17          return a
18      return b
19
```

- ○ None of the others
- ○ T(N)=T(N/2)+C
- ○ T(N)=2*T(N-1)+C
- ◉ T(N)=T(N+1)+C
- ○ T(N)=2*T(N/2)+C



CAN be solved by Master Theorem
T(n/2)+n = 1T(n/2)+n → a>=1, b>1

CANNOT be solved by Master Theorem
T(n/2)+n = 2T(n/1)+n → a>=1, b NOT > 1

MASTER THEOREM

$$T(n) = aT(n/b) + f(n)$$

$a \geq 1$ and $b > 1$ — Condition to be solved by Master Theorem

Case 1:
$f(n) < n^{\log_b a} \Longrightarrow T(n) = \Theta(n^{\log_b a})$

Case 2:
$f(n) = n^{\log_b a} \Longrightarrow T(n) = \Theta(n^{\log_b a} \lg n)$

Case 3:
$f(n) > n^{\log_b a}$ and $a f(n/b) \leq c f(n)$, $c<1$ and n large $\Longrightarrow T(n) = \Theta(f(n))$

EXAMPLE

$$T(n) = 2T(n/2) + n$$

$a=2 \geq 1, b=2>1$ — Condition Fulfilled (can be solved by Master Theorem)

$(a=2, b=2)$

Case 1	Case 2
Is $f(n) < n^{\log_b a}$	Is $f(n) = n^{\log_b a}$
Is $f(n) < n^{\log_2 2}$	Is $f(n) = n^{\log_2 2}$
Is $n < n^1$	Is $n = n^1$
➔ NO	➔ YES

Case 2:
$n = n \Longrightarrow T(n) = \Theta(n \lg n)$

UNIVERSITY OF LONDON

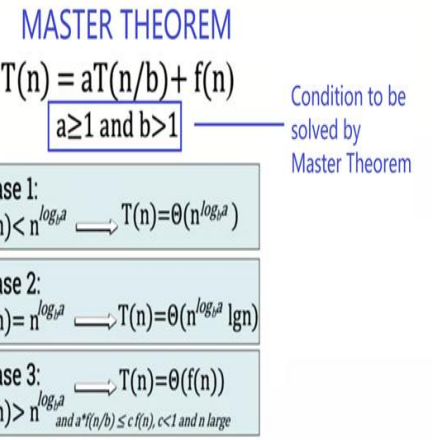
BSc Computer Science

CM2035 Algorithms and Data Structures 2

Quiz

1. The solution to the recurrence equation T(N)=T(N/2)+1 is:

- ☐ $\Theta(N^2)$
- ☐ $\Theta(N^2 \log(N))$
- ☐ $Theta(log(N))$
- ☐ None of the others
- ☐ $\Theta(N \log(N))$
- ☐ $\Theta(N \log^2(N))$
- ☐ $\Theta(N)$



Step 1:

Is it Solvable by Master Theorem ?

T(n) = **a**T(n/**b**) + f(n)

Is a>=1 and b>1 ?

T(N) = **1**T(N/**2**)+1

T(N) = **1**T(N/**2**)+1

Is a>=1 and b>1 ?

(a=1 and b=2)

→ YES

Step 2:

Is it Case 1,2 or 3?

T(n) = **a**T(n/**b**) + f(n)

T(N) = **1**T(N/**2**)+1

Case 1 —- f(n) < n^ (log **b** **a**)

Is f(n) < n ^ (log **b** **a**)

Is f(n) < n ^ (log **2** **1**)

Is f(n) < n ^ (log **2** **2^0**)

Is 1 < n^0

Is 1 < 1

-> NO

Case 2 —--- Is f(n) = n^ (log **b** **a**)

Is f(n) = n ^ (log **b** **a**)

Is f(n) = n ^ (log **2** **1**)

Is f(n) = n ^ (log **2** **2^0**)

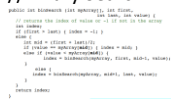
Is 1 = n^0

Is 1 = 1

-> YES

= Theta (n^0 lg n)

= Theta (lgn)

Algorithm	https://www.cs.colostate.edu/~cs200/Fall16/slides/10-divCo.pdf	
//Binary Search 	For binary search $f(n) = a\,f(n/b) + c\,.n^d$ $= 1\,f(n/2) + c$	Binary Search using the Master Theorem For binary search $f(n) = a\,f(n/b) + c\,.n^d$ $= 1\,f(n/2) + c$ $f(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$ Therefore, $d = 0$ (to make n^d a constant), $b = 2$, $a = 1$. $b^d = 2^0 = 1$ It satisfies the second condition of the Master theorem. So, $f(n) = O(n^d \log_2 n) = O(n^0 \log_2 n) = \mathbf{O(\log_2 n)}$
// MergeSort 	*Master theorem* $M(n) = 2M(n/2) + c.n$ for the mergesort algorithm $f(n) = a\,f(n/b) + c.n^d$ $= 2\,f(n/2) + c.n^1$	Complexity of MergeSort *Master theorem* $M(n) = 2M(n/2) + c.n$ for the mergesort algorithm $f(n) = a\,f(n/b) + c.n^d$ $= 2\,f(n/2) + c.n^1$ $f(n) = \begin{cases} O(n^d) & \text{if } a < b \\ O(n^d \log n) & \text{if } a = b \\ O(n^{\log_b a}) & \text{if } a > b \end{cases}$ Notice that c does not play a role(big O) $d = 1$, $b = 2$, $a = 2$. Therefore $b^d = 2^1 = 2$ It satisfies the second condition of the Master theorem. So, $f(n) = O(n^d \log_2 n)$ $= O(n^1 \log_2 n)$ $= \mathbf{O(n \log_2 n)}$

// Sum returns the sum 1 + 2
// + ... + n, where n >= 1.
 	
func Sum(n int) int {
  if n == 1 {
  return 1
  }
  return n + Sum(n-1)
}
 	
https://yourbasic.org/algorithms/time-complexity-recursive-functions/
	T(1) = 1, (*)
	
	T(n) = 1 + T(n-),
	when $n > 1$
		To find the time complexity for the Sum function can then be **reduced to solving the recurrence relation**
		T(1) = 1,
		T(n) = 1 + T(n-1), when $n > 1$.
		--- 	
		By repeatedly applying these relations, we can compute T(n) for any positive number n.
		T(n) = (**)
		1 + T(n-1) = (**)
		1 + (1 + T(n-2)) = 2 + T(n-2) = (**)
		2 + (1 + T(n-3)) = 3 + T(n-3) = …
		k + T(n-k) = …
		n - 1 + T(1) = (*)
		n - 1 + 1
		= **Θ(n)** **(Theta Average Case)**

https://www.cs.colostate.edu/~cs200/Fall16/slides/10-divCo.pdf

https://yourbasic.org/algorithms/time-complexity-recursive-functions/

```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
#include <string>
using namespace std;

int M[2][2] = {  {2, -5},
                 {4, 0 }
              };
int N = 2;
int G = 0;     // Goldie
string s ;

string FIND_GOLDIE (int M[2][2], int N, int G )
{
  for (int i = 0; i <= 1; ++i)
  {
    for (int j = 0; j <=1; ++j)
    {
      cout << M[i][j]<<" ";
      if (M[i][j] == G)
      { s =  "(" + to_string(i) + "," + to_string(j) + ")";
        return s;
      }
    }
    cout << endl;
  }
  return "(-1,-1)";
}


int main()
{
  cout << FIND_GOLDIE (M, N, G);
  return 0;
}
```

ALGORITHMS & DATA STRUCTURES 2
Mid-Term Coursework
Where's Goldie?

1. INTRODUCTION

Where's Goldie? is a search game. But, instead of finding a person in a page filled with hundreds of drawings of people doing amusing things, you must a find a number (Goldie) in a **square matrix** of integers made of N rows and N columns. The range of the integers stored in the matrix goes from 0 to 500.

No number is repeated in the matrix. The number representing Goldie is entered as an input parameter (G) to the function FIND_GOLDIE(M, N, G), where M is the matrix, N the number of rows (or columns) and G the number to search for.

We have received 3 algorithmic proposals to do this. They are as follows.

Proposal 1: You must visit every element in the square matrix. If the element you are visiting is equal to G, then you have found Goldie! In that case, you return the position of Goldie as a pair (row, column). If, after visiting every element of the matrix, you did not find Goldie, you return the pair (-1,-1).

Proposal 2: Before visiting the elements of the square matrix, you process its content by sorting the elements of each row from smallest to largest. Figure 2 shows an example of the original matrix (left) and how its content is rearranged after sorting every row (right).

	[0]	[1]	[2]	[3]	[4]
[0]	3	23	14	6	18
[1]	97	32	12	75	49
[2]	25	11	48	4	10
[3]	5	98	7	21	31
[4]	1	46	34	9	17

⇒

	[0]	[1]	[2]	[3]	[4]
[0]	3	6	14	18	23
[1]	12	32	49	75	97
[2]	4	10	11	25	48
[3]	5	7	21	31	98
[4]	1	9	17	34	46

Figure 2. Original matrix M (left) and row-by-row processed matrix (right).

```
// https://www.onlinegdb.com/online_c++_compiler
#include <iostream>
#include <cmath>
using namespace std;

int Find (int key, int B[], int M, int j)
{  for (int i = 0; i < M; i++)
      if (B[i] == key)
        return j + 2*i;
   return -1;
}

int R0(int key, int A1[], int A2[], int N)
{  for (int i = 0; i < ceil (N/2); i++)
      if (A1[i] == key)
        return 2*i;
   ////???

}

int main()
{
   int A[4]  = { 7,5,6,3 };
   int A1[2];
   int A2[2];
   int i=0;
   int j=0;


   for (int x = 0; x <= 3; x++)
   {
      if (x % 2 == 0)
      {  A1[i] = A[x];
         i++;
      }
      if (x % 2 == 1)
      {  A2[j] = A[x];
         j++;
      }

   }

   cout << "\n Array A \n";
   for (int y = 0; y <= 3; y++)
   {
      cout << A[y];
   }

   cout << "\n Array A1 \n";
   for (int y = 0; y <= 1; y++)
   {
      cout << A1[y];
   }

   cout << "\n Array A2 \n";
   for (int y = 0; y <= 1; y++)
   {
      cout << A2[y];
   }
   cout<<endl;

   cout << "Find returns : " <<Find (5,A,4,2)<<endl;
   cout << "R0 returns : " <<R0 (5,A1,A2,2)<<endl;
   return 0;
```

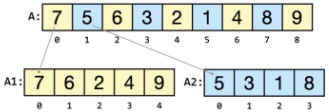
ALGORITHMS & DATA STRUCTURES 2

Mid-Term Assignment

Houston, we have an error!

3. THE FIRST PART: THE R0 SEARCH ALGORITHM

The first scenario we consider is inspired by RAID 0. In this scenario we start with an array A of length N storing non-negative integers, and create two new arrays where the first array stores the values `A[i]` where `i` is an even number, and the second array stores the values `A[j]` where `j` is an odd number. The following diagram shows this scenario:



There are now two new arrays, A1 and A2, are created and the values at even-numbered indices of A are copied to A1, and the values at odd-numbered indices are copied to A2.

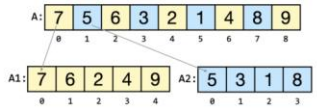
The idea is now that instead of storing A, we store two separate arrays in a distributed manner. However, if we want to search the array A for particular values, we now need to search these two arrays and then relate the indices back to the original array. The next two tasks together give such a method to search the two arrays in this scenario.

Task 1: Consider the following pseudocode function that describes the **R0 Search** algorithm:

```
function R0(key,A1,A2,N)
     for 0 <= i < ceiling(N/2)
          if(A1[i] == key):
               return 2*i
     for 0 <= i < floor(N/2)
          if(A2[i] == key):
               return 2*i + 1
     return -1
```

3. THE FIRST PART: THE R0 SEARCH ALGORITHM

The first scenario we consider is inspired by RAID 0. In this scenario we start with an array A of length N storing non-negative integers, and create two new arrays where the first array stores the values `A[i]` where `i` is an even number, and the second array stores the values `A[j]` where `j` is an odd number. The following diagram shows this scenario:



In this diagram we see that two new arrays, A1 and A2, are created and the values at even-numbered indices of A are copied to A1, and the values at odd-numbered indices are copied to A2.

The idea is now that instead of storing A, we store two separate arrays in a distributed manner. However, if we want to search the array A for particular values, we now need to search these two arrays and then relate the indices back to the original array. The next two tasks together give such a method to search the two arrays in this scenario.

Task 1: Consider the following pseudocode function:

```
function Find(key,B,M,j)
     for 0 <= i < M
          if(B[i] == key):
               return j + 2*i
     return -1
```